Lab 11 — Model Compression Benchmark

Track B (BMD-45 vehicle detection) · Week 11 · DS5619 Machine Learning Systems Operations

Setup

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python generate_for_student.py --student-id <your roll number or institute email>
```

This overwrites `data/weights.json` and `data/eval_data.json` with your own values, seeded deterministically from your student ID — same structure as everyone else's (256 weights, 40 informative), different actual numbers. The ~17x magnitude gap between informative and noise weights is a structural guarantee of how the generator builds them, not a matter of luck, so the "safe at 70% sparsity, cliff at 95%" story holds for every student ID.

Record your `--student-id` value in `NOTES.md`. The grader re-runs `generate_for_student.py` with the ID you recorded and diffs the result against what you committed.

Files

- `src/compression.py` — implement the five `# TODO` functions.
- `src/run_pipeline.py` — complete driver script. Don't edit.
- `prep_data.py` — superseded, don't run (see `generate_for_student.py`).

Background

`data/weights.json` holds 256 weights for a linear classifier — a stand-in for a detector's classification head. It's deliberately over-parameterized: only 40 of the 256 weights are actually informative (large magnitude, correlated with the label), the other 216 are near-zero noise. This is the same "vastly over-specified" scenario the lecture opens with. `data/eval_data.json` has 300 labeled examples generated FROM these exact weights, so the unmodified model should score close to 100%. Whatever accuracy you lose after compressing is coming from the compression itself, not from noise in the data.

A note on the latency numbers you'll see: this reference implementation is pure Python, and `predict()` runs on dequantized floats for every variant — including the INT8 one, where `quantize_int8`/`dequantize_int8` convert back to floats before any arithmetic happens. That means `latency_ms_per_pass` will come out roughly the same across all four variants here. That's an artifact of this teaching implementation, not a real-world result — in a production serving stack, INT8 inference runs on dedicated integer kernels and is genuinely faster, which is the whole reason INT8 quantization is worth doing. Don't read flat latency numbers in your `compression_report.json` as "quantization failed to speed anything up"; the size and accuracy columns are where this lab's real signal is.

Your task

`src/compression.py` (five functions marked `# TODO`, each has a full docstring spec)

- `model_size_bytes(weights, dtype_bytes)` — storage size for a given bytes-per-parameter.
- `prune_weights(weights, sparsity)` — zero out the smallest-magnitude fraction of weights.
- `sparse_coo_size_bytes(weights)` — bytes needed to store only the non-zero weights (value + index each): the sparse side of the pruning pitfall.
- `quantize_int8(weights)` — symmetric linear quantization to signed 8-bit integers.
- `benchmark_model(label, weights, eval_examples, dtype_bytes)` — size, accuracy, and latency for one weight set.

```
python src/run_pipeline.py
```

Benchmarks four variants: baseline FP32, 70%-pruned-but-still-dense FP32 (the pitfall — same size as baseline), the same pruning stored sparsely (actually smaller), and INT8 quantized. Writes `compression_report.json` (`src/run_pipeline.py` is complete, don't edit it).

Self-check

```
pytest tests/ -q
```

This is a self-check, not the grader.

Deliverables (what you commit)

- `src/compression.py`, completed.
- `compression_report.json` (from running the pipeline).
- A short `NOTES.md`: the `--student-id` value you used (required — see above), and compare `pruned_dense_fp32` and `pruned_sparse_coo` in your report — same weights, same accuracy, very different `size_bytes`. Explain why in your own words (tie it to the lecture's pruning pitfall). Then try editing `PRUNE_SPARSITY` in `src/run_pipeline.py` to `0.95` and re-run — what happens to accuracy, and why does that make sense given only 40 of the 256 weights are actually informative?

Grading checklist

- [ ] `data/weights.json` and `data/eval_data.json` match what `generate_for_student.py --student-id <NOTES.md value>` actually produces.
- [ ] `prune_weights` correctly identifies the smallest-magnitude weights and doesn't mutate its input (checked with a held-out weight vector).
- [ ] `quantize_int8` uses scale = max|w| / 127, maps the largest-magnitude weight to exactly +/-127, rounds to nearest, handles all-zero input (scale 1.0), and round-trips through `dequantize_int8` with error of at most half a quantization step.
- [ ] `sparse_coo_size_bytes` counts value + index bytes for non-zero weights only (0 for all-zero input).

- [ ] `benchmark_model` computes accuracy and size correctly; latency is measured and averaged over the repetitions, not hardcoded.
- [ ] The real pipeline run shows `pruned_dense_fp32` and `pruned_sparse_coo` with identical accuracy but different `size_bytes` — that gap IS the point of this lab.
- [ ] `NOTES.md` correctly explains the dense-vs-sparse gap and reports the 0.95-sparsity accuracy-cliff experiment.
- [ ] Meaningful commit history and a working README.

Submission

```
git add -A
git commit -m "Week 11: model compression benchmark"
git tag week11-submit
git push origin main --tags
```